

A Project Report
On
Predictive Mental Health Monitoring using Social
Media Data

*Submitted in partial fulfilment of the
requirement for the award of the degree of*

MASTER OF COMPUTER APPLICATIONS



(Established under Galgotias University Uttar Pradesh Act No. 14 of 2011)

Session 2025-26
in
Master of Computer Applications

by
Deepak Singh Kushwaha (24SCSE2240094)
Mohd. Zishan (24SCSE2240062)
Saif Hussain (24SCSE2240113)

Under the guidance of
Mr. Anshuman

Assistant Professor II (SCAT)
(GUSCSE202332640)

SCHOOL OF COMPUTER APPLICATIONS AND TECHNOLOGY

GALGOTIAS UNIVERSITY, GREATER NOIDA

INDIA

June, 2026



**SCHOOL OF COMPUTER APPLICATIONS AND
TECHNOLOGY**
GALGOTIAS UNIVERSITY, GREATER NOIDA

CANDIDATE'S DECLARATION

We hereby certify that the work which is being presented in the project, entitled “**Predictive Mental Health Monitoring using Social Media Data**” in partial fulfilment of the requirements for the award of the MCA (Master of Computer Applications) submitted to the School of Computer Applications and Technology, Galgotias University, Greater Noida, is an original work carried out during the period of February, 2026 to June 2026, under the supervision of **Mr. Anshuman**, Assistant Professor, Department of Computer Science and Technology, Galgotias University, Greater Noida.

The matter presented in the project has not been submitted by us for the award of any other degree of this or any other places.

Deepak Singh Kushwaha

Mohd. Zishan

Saif Hussian

CERTIFICATE

This is to certify that Project Report entitled “**Predictive Mental Health Monitoring using Social Media Data**” which is submitted by Deepak Singh, Mohd. Zishan and Saif Hussain in partial fulfilment of the requirement for the award of degree MCA. in School of Computer Applications and Technology, Galgotias University, Greater Noida, India is a record of the candidate own work carried out by them under my supervision. The matter embodied in this thesis is original and has not been submitted for the award of any other degree

Signature of Examiner(s)

Signature of Supervisor(s)

Date: June, 2026

Place: Greater Noida

ABSTRACT

This project presents a comprehensive and scalable system for predictive mental health monitoring by leveraging machine learning, natural language processing, cloud deployment, and mobile application integration. The system is designed to analyze user-generated data, particularly text and behavioral patterns, to identify the most dominant mental health disorder affecting an individual. With the increasing use of digital platforms where users frequently express emotions, thoughts, and experiences, such data serves as a valuable source for early detection of psychological conditions.

The work begins with the collection of multiple datasets representing various mental health conditions such as depression, anxiety, stress, bipolar disorder and suicidal tendencies. These datasets include both structured and unstructured formats, enabling the system to capture a wide range of indicators. Extensive preprocessing techniques are applied, including data cleaning, normalization, and transformation, along with Natural Language Processing methods such as tokenization, stop-word removal, lemmatization, and TF-IDF vectorization. These steps ensure that the data is converted into meaningful features suitable for machine learning models.

Multiple machine learning algorithms are implemented, including Logistic Regression, Random Forest, XGBoost, LightGBM, CatBoost, Support Vector Machine, and Neural Networks. Each model is trained and evaluated using standard performance metrics such as accuracy, precision, recall, F1-score, and ROC-AUC. Comparative analysis is conducted to identify the best-performing model, ensuring both accuracy and efficiency. The selected models demonstrate high predictive performance, achieving accuracy levels exceeding 90% across different datasets.

A key contribution of this system is the transformation of trained models from PKL format into ONNX format. This conversion enables platform independence, faster inference, and reduced dependency on specific programming environments. The ONNX models are then deployed on the Hugging Face platform, which provides scalable cloud infrastructure and API-based access to the models. This deployment allows real-time interaction between the backend system and external applications.

To enhance usability, a mobile application is developed using Android Studio. The application serves as the user interface, allowing individuals to input their data

and receive predictions in real time. The app communicates with the deployed API by sending user input and receiving prediction results, which indicate the most affected mental health disorder. This seamless integration of backend intelligence and frontend accessibility ensures that the system is practical and user-friendly.

The overall system follows a structured workflow that includes data collection, preprocessing, model training, evaluation, conversion, deployment, and application integration. This step-by-step approach ensures scalability, modularity, and maintainability. The system is capable of delivering fast and accurate predictions while maintaining efficiency through optimized deployment strategies.

The results demonstrate that ensemble models such as XGBoost and CatBoost provide superior performance due to their ability to capture complex patterns in data. The use of ONNX significantly improves inference speed, while cloud deployment ensures high availability and scalability. The Android application successfully provides real-time predictions, making the system accessible to end users.

This project highlights the potential of artificial intelligence in supporting mental health monitoring and early detection. While the system is not intended to replace professional diagnosis, it serves as a powerful tool for awareness and preliminary assessment. The modular design allows future enhancements such as integration with real-time social media data, advanced deep learning models, personalized recommendations, and improved visualization dashboards.

In conclusion, the project successfully demonstrates the transformation of machine learning models into a fully deployed, real-time application. It bridges the gap between research and practical implementation, providing a scalable, efficient, and user-friendly solution for predictive mental health monitoring.

TABLE OF CONTENTS

CANDIDATE'S DECLARATION	ii
CERTIFICATE.....	iii
ABSTRACT	iv
LIST OF TABLES	vii
LIST OF FIGURES	vii
LIST OF ABBREVIATIONS	vii
CHAPTER 1 – INTRODUCTION	1
1.1 Problem Introduction	1
1.2 Motivation	1
1.3 Project Objectives	2
1.4 Scope of the Project.....	2
1.5 Related Previous Work (Brief)	3
CHAPTER 2 – SOFTWARE REQUIREMENT SPECIFICATION (SRS)	4
2.1 Product Perspective.....	4
2.1.1 System Interfaces	6
2.1.2 Interfaces (User Interfaces)	6
2.1.3 Hardware Interfaces.....	6
2.1.4 Software Interfaces	7
2.1.5 Communications Interfaces	8
2.1.6 Memory Constraints.....	8
2.1.7 Operations	8
2.1.8 Site Adaptation Requirements	9
2.2 Product Functions	9
2.3 User Characteristics	9
2.4 Constraints	9
2.5 Assumptions and Dependencies	9
2.6 Apportioning of Requirements	9
2.7 Use Case Model	9

CHAPTER 3 – SYSTEM DESIGN	11
3.1 Overall Architecture	11
3.2 Data Flow Diagram (Narrative)	12
3.3 Module Design	15
3.4 Database / Storage Design	17
 CHAPTER 4 – IMPLEMENTATION AND RESULTS	 19
4.1 Software and Hardware Requirements.....	19
4.2 Assumptions and Dependencies	20
4.3 Implementation Details	20
4.4 Test Cases.....	22
4.5 Results and Discussion.....	23
 CHAPTER 5 – CONCLUSION AND FUTURE WORK.....	 25
5.1 Conclusion.....	25
5.2 Performance Evaluation	26
5.3 Comparison with State-of-the-Art.....	27
5.4 Future Directions	28
 APPENDIX A – SOURCE CODE	 30
A.1 Machine Learning Model Development.....	30
A.2 Model Hosting on Hugging Face	47
A.3 Inference API Implementation	48
A.4 Kotlin Android Client Integration	49

LIST OF TABLES

• <i>Table 1 Model Performance</i>	5
• <i>Table 2 Tools and Platforms Used for model</i>	5
• <i>Table 3 Tools and Platforms Used for Anroid App</i>	5
• <i>Table 4 Hardware Interfaces</i>	7
• <i>Table 5 Software Interfaces</i>	7
• <i>Table 6 Model Performance Comparison</i>	21
• <i>Table 7 Test Case Table</i>	23
• <i>Table 8 Performance Summary</i>	26
• <i>Table 9 Comparison with Existing Systems</i>	28

LIST OF FIGURES

• <i>Figure 1 Flow Representation</i>	7
• <i>Figure 2 Product Function Lifecycle</i>	8
• <i>Figure 3 Primary Use Case Model</i>	9
• <i>Figure 4 Secondary Use Case Model</i>	10
• <i>Figure 5 Full Architecture Flow</i>	12
• <i>Figure 6. DFD Level 0</i>	13
• <i>Figure 7. DFD Level 1</i>	13
• <i>Figure 8. DFD Level 2</i>	14
• <i>Figure 9. Functional Module Design of the Proposed System</i>	16

LIST OF ABBREVIATIONS

- ML – Machine Learning
- NLP – Natural Language Processing
- EDA – Exploratory Data Analysis
- ROC – Receiver Operating Characteristic
- AUC – Area Under the Curve
- TF-IDF – Term Frequency–Inverse Document Frequency
- CSV – Comma Separated Values
- GUI – Graphical User Interface

CHAPTER 1 – INTRODUCTION

1.1 Problem Introduction

In recent years, the rapid growth of social media platforms has transformed the way individuals express their thoughts, emotions, and daily experiences. Platforms such as Reddit, Twitter, Instagram, and Facebook have become digital environments where users openly share their mental states, personal struggles, and behavioural patterns. This continuous stream of user-generated content provides a valuable data source for analyzing psychological conditions using computational methods.

Mental health disorders such as depression, anxiety, stress, bipolar disorder and suicidal ideation are increasing at a significant rate globally. Traditional methods of diagnosis rely on clinical observations and self-reporting, which often fail to capture real-time emotional changes or early warning signals. Therefore, there is a strong need for an automated system that can continuously monitor and analyze behavioral and textual patterns to detect mental health risks at an early stage.

The developed system addresses this problem by leveraging machine learning and natural language processing techniques to analyze social media-like data and predict the most dominant mental health disorder affecting a user. The system evolves from a data-driven model pipeline into a fully functional application capable of delivering real-time predictions.

1.2 Motivation

The motivation behind this work arises from the increasing digital presence of individuals and the rich psychological signals embedded in their online activities. People often express emotions such as sadness, stress, loneliness, or anxiety through posts, comments, and interactions. These expressions, when analyzed correctly, can provide meaningful insights into their mental health condition.

Initially, the work focused on building a strong machine learning pipeline capable of handling multiple datasets and predicting different disorders with high accuracy. However, having a trained model alone is not sufficient for real-world impact. There was a need to transform this model into a usable system that could interact with users and provide real-time insights.

This led to the motivation for extending the work into a complete application by integrating model deployment, API communication, and mobile application development. The goal was to create an end-to-end intelligent system that bridges the gap between machine learning models and real-world usability.

1.3 Project Objectives

The primary objectives of this work are structured to reflect the transition from model development to full system deployment:

- To develop a robust machine learning pipeline capable of processing textual and structured data for predicting multiple mental health disorders.
- To train and evaluate multiple models such as Logistic Regression, Random Forest, Gradient Boosting, XGBoost, LightGBM, CatBoost, and Neural Networks using appropriate evaluation metrics.
- To convert trained machine learning models from PKL format to ONNX format to ensure platform independence, faster inference, and better deployment compatibility.
- To deploy the ONNX models on a cloud platform (Hugging Face) and expose them through APIs for real-time access.
- To design and develop an Android application using Android Studio that interacts with the deployed API and displays prediction results to the user.
- To create a complete workflow where the mobile application sends user data to the API and receives predictions indicating the most affected mental health disorder.
- To build a scalable and modular system that can be extended with additional features such as real-time monitoring, advanced models, and personalized recommendations.

1.4 Scope of the Project

The scope of this work covers the complete lifecycle of a machine learning-based application, starting from data processing to real-time deployment and user interaction.

In the initial phase, the work focuses on dataset handling, preprocessing, feature engineering, and training multiple machine learning models for different mental health conditions. This includes handling both structured and textual data using NLP techniques and statistical methods.

In the extended phase, the scope expands to include model conversion, deployment, and application integration. The trained models are converted into ONNX format to ensure compatibility with different platforms. These models are then deployed on Hugging Face, where they are exposed as APIs.

The Android application serves as the front-end interface, allowing users to interact with the system. The app sends requests to the API, receives predictions, and displays the disorder that most significantly affects the user.

The system is designed to be scalable and adaptable, allowing future enhancements such as integration with live social media data, real-time monitoring, and advanced deep learning techniques.

1.5 Related Previous Work (Brief)

Previous research in mental health prediction has primarily focused on analyzing social media data using machine learning and natural language processing techniques. Studies have shown that linguistic patterns, sentiment analysis, and behavioral features can be effectively used to detect conditions such as depression and anxiety.

Many existing systems focus on single-condition prediction, such as detecting depression from Twitter data or identifying suicidal ideation from text posts. These systems often use models like Support Vector Machines, Logistic Regression, or deep learning approaches such as LSTM and BERT.

In contrast, this work extends beyond traditional approaches by integrating multiple disorder predictions into a unified system and further transforming the model into a deployable application. The key advancement lies in the transition from offline model development to a real-time, API-driven mobile application.

Additionally, the use of ONNX for model conversion and Hugging Face for deployment introduces a modern, scalable approach to machine learning system design. The integration with an Android application further enhances accessibility and usability, making the system practical for real-world implementation.

This structured progression from model development to deployment and application integration distinguishes the work and highlights its contribution to the field of predictive mental health monitoring.

CHAPTER 2 – SOFTWARE REQUIREMENT SPECIFICATION (SRS)

2.1 Product Perspective

The system is designed as a complete end-to-end intelligent solution that evolves progressively from raw data processing to real-time application deployment. Initially, the work begins with collecting multiple datasets related to mental health conditions such as depression, anxiety, stress, bipolar disorder and suicidal tendencies. These datasets include both structured (tabular) and unstructured (text) formats, making the system capable of handling diverse data sources.

In the first stage, extensive data preprocessing is performed. This includes handling missing values, removing duplicates, normalizing data, and applying Natural Language Processing (NLP) techniques such as tokenization, stop-word removal, lemmatization, and TF-IDF vectorization. Feature engineering is then applied to extract meaningful patterns such as emotional intensity, sentiment polarity, and behavioral indicators.

After preprocessing, multiple machine learning models are trained including Logistic Regression, Random Forest, Gradient Boosting, XGBoost, LightGBM, CatBoost, Support Vector Machine, and Multi-Layer Perceptron (MLP). These models are evaluated using standard performance metrics such as Accuracy, Precision, Recall, F1-Score, and ROC-AUC. Based on comparative analysis, the best-performing models are selected.

S. No.	Model	Accuracy	Precision	Recall	F1-Score
1.	Logistic Regression	0.935	0.938	0.935	0.934
2.	Random Forest	0.965	0.965	0.964	0.964
3.	XGBoost	0.949	0.952	0.949	0.979
4.	LightGBM	0.947	0.951	0.947	0.947

Table 1 Model Performance

In the next stage, the trained models stored in PKL format are converted into ONNX format. This conversion is important because ONNX provides cross-platform compatibility, faster inference, and reduced dependency on Python environments. It enables smooth integration with mobile and cloud-based systems.

Following this, the ONNX models are deployed on Hugging Face, which provides a scalable cloud environment and API endpoints for accessing the models. Finally, an Android application is developed using Android Studio. The application communicates with the deployed API, sends user input, and receives predictions indicating the most affected mental health disorder.

Tools and Platforms Used for model Building:

S. No.	Category	Details
1.	Programming Language	Python
2.	Libraries	pandas, numpy, scikit-learn, xgboost, lightgbm, catboost, nltk
3.	Model Format	PKL → ONNX
4.	Deployment Platform	Hugging Face
5.	Model Development	Sk-learn Package
6.	API Communication	REST API

Table 2 Tools and Platforms Used

Tools and Platforms Used for Android App Building:

S. No.	Category	Details
1.	Programming Language	Kotlin
2.	Libraries	Jetpack Compose, Ktor, Koin, Kotlinx Serialization
3.	Model Format	Model → Result
4.	Deployment Platform	Hugging Face
5.	Mobile Development	Android Studio
6.	API Communication	REST API

Table 3 Tools and Platforms Used

2.1.1 System Interfaces

The system integrates multiple interfaces across different stages. Initially, it interfaces with CSV datasets for input. After preprocessing, it interacts with trained machine learning models stored locally. During deployment, the system connects with Hugging Face servers where ONNX models are hosted.

The Android application acts as a client interface, sending input data via API requests and receiving predictions. This ensures seamless communication between backend and frontend components.

2.1.2 Interfaces (User Interfaces)

The Android application serves as the primary user interface of the proposed system. It is designed using Android Studio with a focus on simplicity, responsiveness, and ease of use so that users with minimal technical knowledge can interact with the system effectively.

The application provides a clean and intuitive interface through which users can enter text-based information related to their emotions, thoughts, feelings, or social media posts. The entered data is processed through a cloud-hosted machine learning model, and the prediction results are displayed in real time. The user does not need any knowledge of machine learning or data analysis, as all complex processing is performed in the backend.

The application follows Material Design principles to ensure consistency, readability, and accessibility across different Android devices.

Communication between the Android application and the deployed Hugging Face API occurs through RESTful HTTP requests. When the user submits input data, the application sends a request to the cloud-hosted model. The prediction response is then received, parsed, and displayed within the application interface.

2.1.3 Hardware Interfaces

The system operates on standard hardware configurations. Development is performed on a laptop or desktop, while deployment is cloud-based. The Android application runs on smartphones.

S. No.	Component	Specification
1.	Processor	Intel i5 or above
2.	RAM	4GB or higher
3.	Storage	20GB free space
4.	Internet Connectivity	Required

Table 4 Hardware Interfaces

2.1.4 Software Interfaces

The mobile application uses Android Studio, Android SDK, XML-based layouts, Java or Kotlin for activity logic, Gradle for build configuration, and an HTTP client with JSON parsing for API communication.

The Android application communicates with the deployed Hugging Face API through HTTPS requests. The user input is sent as JSON, and the response returns the predicted label and confidence value, which are parsed and displayed in the app.

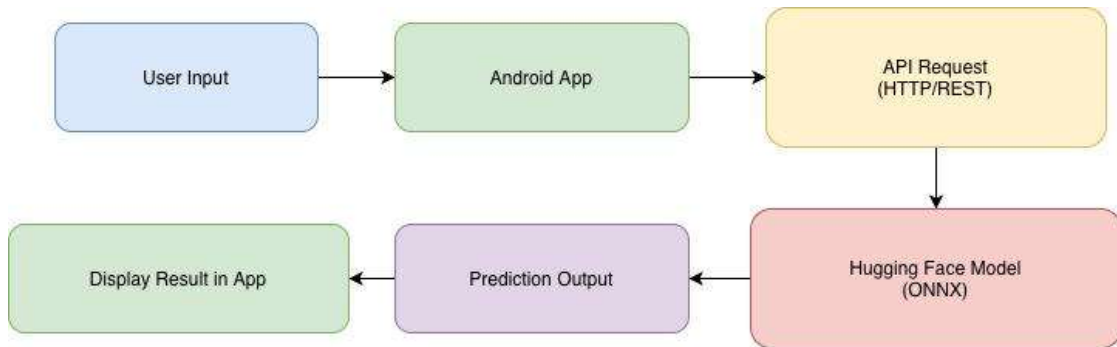


Figure 1 Flow Representation

The system uses a combination of software tools and frameworks:

S. No.	Tool / Platform	Purpose
1.	Python	Backend development
2.	Jupyter Notebook / Colab	Experimentation and model training
3.	ONNX Runtime	Model execution
4.	Hugging Face	Deployment and API hosting
5.	Android Studio	Mobile application development

Table 5 Software Interfaces

2.1.5 Communications Interfaces

Communication is handled through REST APIs. The Android application sends HTTP requests to the Hugging Face API, and the server returns prediction results.

Flow Representation:

2.1.6 Memory Constraints

The system is optimized for efficient memory usage. During training, higher memory is required for handling datasets. However, ONNX models reduce memory usage during inference. The Android app is lightweight as processing is handled on the server side.

2.1.7 Operations

The system operates in the following sequence:

1. Data Collection
2. Data Preprocessing
3. Feature Engineering
4. Model Training
5. Model Evaluation
6. Model Conversion (PKL → ONNX)
7. Model Deployment (Hugging Face)
8. API Integration
9. Android Application Interaction

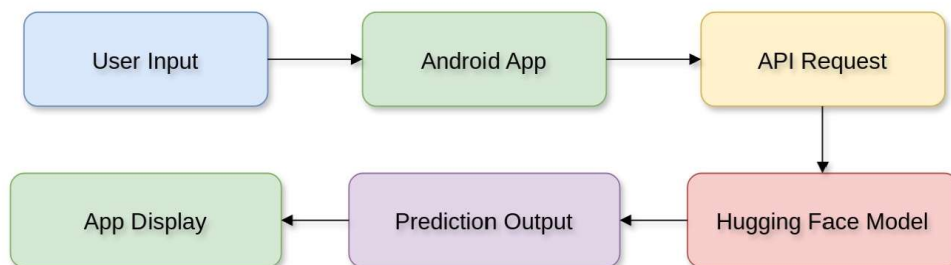


Figure 2 Product Functions Lifecycle

This structured workflow ensures reliability and scalability.

2.1.8 Site Adaptation Requirements

The system can be adapted easily by modifying dataset paths, API endpoints, and configuration settings. It supports scalability and can be extended to include real-time data sources.

2.2 Product Functions

The system performs multiple functions across its lifecycle:

2.3 User Characteristics

The system is designed for both technical and non-technical users. General users can interact through the Android app, while developers can modify backend components.

2.4 Constraints

- Dependence on dataset quality
- Internet required for API access
- Not a medical diagnostic tool

2.5 Assumptions and Dependencies

- Input data is valid
- API is available
- Required libraries are installed

2.6 Apportioning of Requirements

The system is developed in phases:

- Initial Phase: Data processing and model training
- Intermediate Phase: Model conversion and optimization
- Final Phase: Deployment and Android integration

2.7 Use Case Model

Primary Use Case:

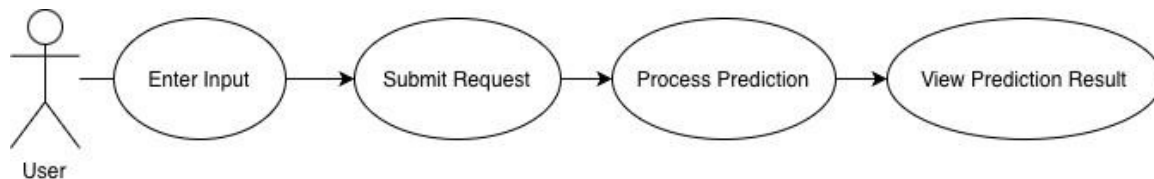


Figure 3 Primary Use Case Model

Secondary Use Case:

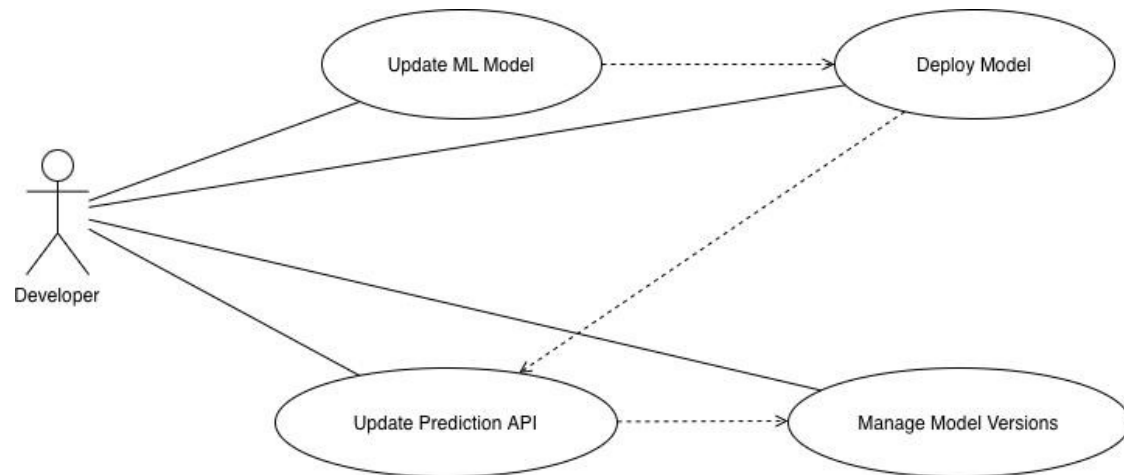


Figure 4 Secondary Use Case Model

CHAPTER 3 – SYSTEM DESIGN

3.1 Overall Architecture

The system follows a multi-layered, scalable architecture that evolves step-by-step from data processing to a fully functional mobile-based prediction system. The design ensures clear separation between data handling, model processing, deployment, and user interaction, making the system modular, maintainable, and extensible.

The architecture is developed in a sequential manner. Initially, the work begins with dataset acquisition and preprocessing. Multiple datasets related to different mental health conditions are collected and stored in CSV format. These datasets include both structured and textual data, which require different processing techniques.

After dataset preparation, the next layer focuses on machine learning model development. Various algorithms are trained and evaluated, and the best-performing models are selected. These models are stored in PKL format as serialized objects.

In the next stage, the architecture transitions towards deployment. The trained PKL models are converted into ONNX format to ensure cross-platform compatibility and optimized inference performance. This step is critical because ONNX allows models to be used outside Python environments and supports integration with mobile and cloud systems.

Following model conversion, the ONNX models are deployed on Hugging Face. This creates an API layer that acts as a bridge between the backend models and the frontend application.

Finally, the Android application is developed, which serves as the user interface. The application sends requests to the API and receives predictions in real time.

Key Components of Architecture:

- Data Layer:
 - Stores raw datasets (CSV files)
 - Stores cleaned and processed datasets
 - Handles input data from users
- Processing Layer:
 - Data preprocessing and NLP operations
 - Feature extraction and transformation
 - Model training and evaluation

- Model Layer:
 - Contains trained ML models
 - Stores models in PKL format
 - Converts models to ONNX
- Deployment Layer:
 - Hosts ONNX models on Hugging Face
 - Provides API endpoints
- Application Layer:
 - Android application interface
 - Sends user input to API
 - Displays prediction results
- Communication Layer:
 - REST API communication
 - Handles request-response cycle

This layered architecture ensures flexibility, scalability, and ease of maintenance. It also allows independent updates to each component without affecting the entire system.

Full Architecture Flow:

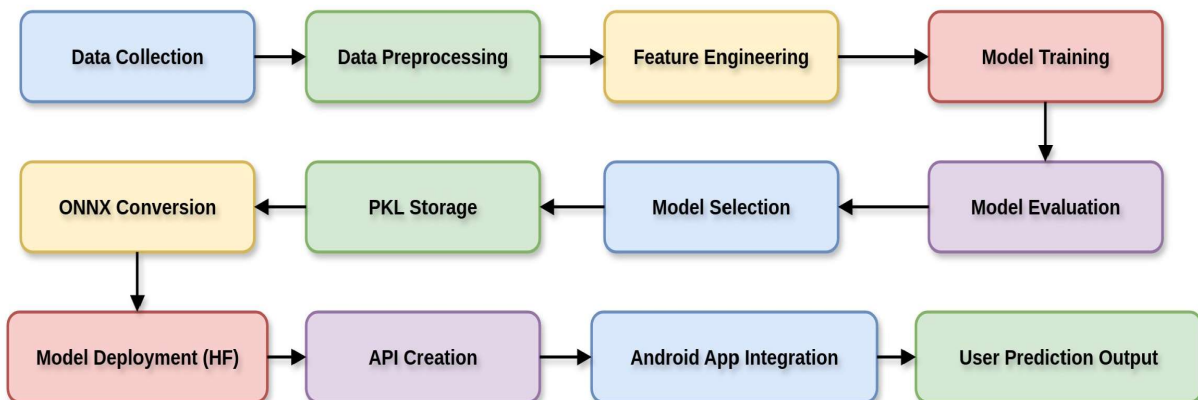


Figure 5 Full Architecture Flow

3.2 Data Flow Diagram

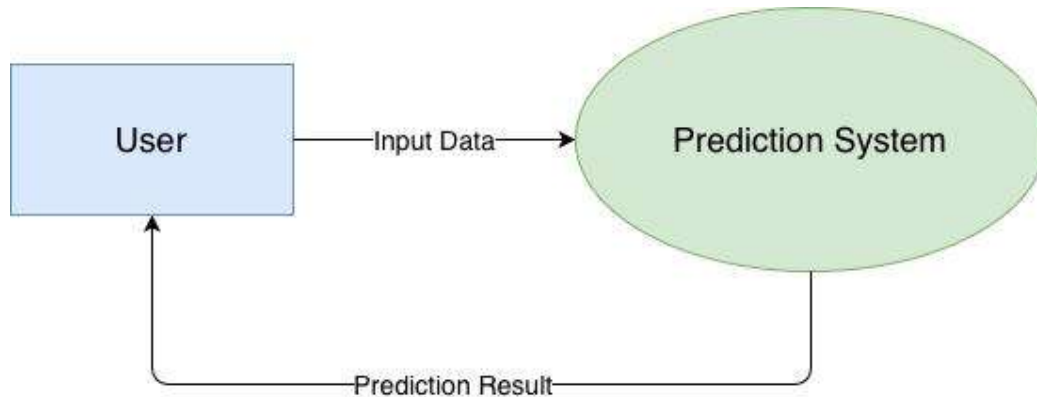
The data flow within the system is structured in a logical sequence, ensuring efficient processing and communication between components.

Initially, raw data is loaded from datasets or user input. This data is passed through preprocessing stages where cleaning, normalization, and transformation occur.

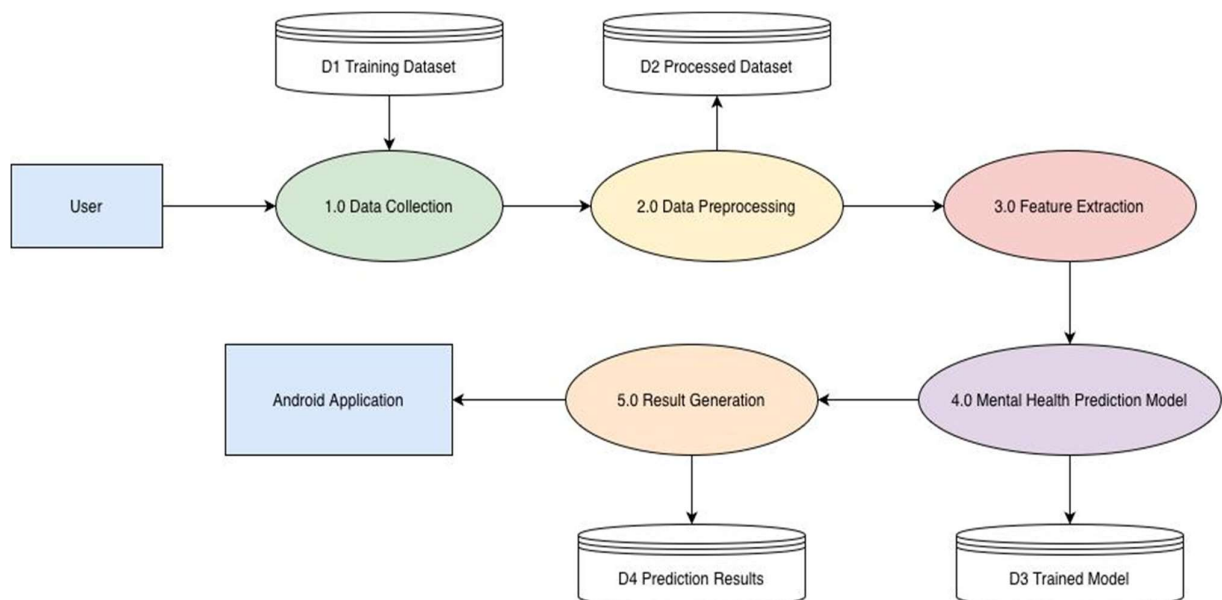
After preprocessing, features are extracted and passed to machine learning models for training and prediction.

Once models are trained, predictions are generated and stored. In the deployment phase, the same flow is adapted for real-time predictions through APIs.

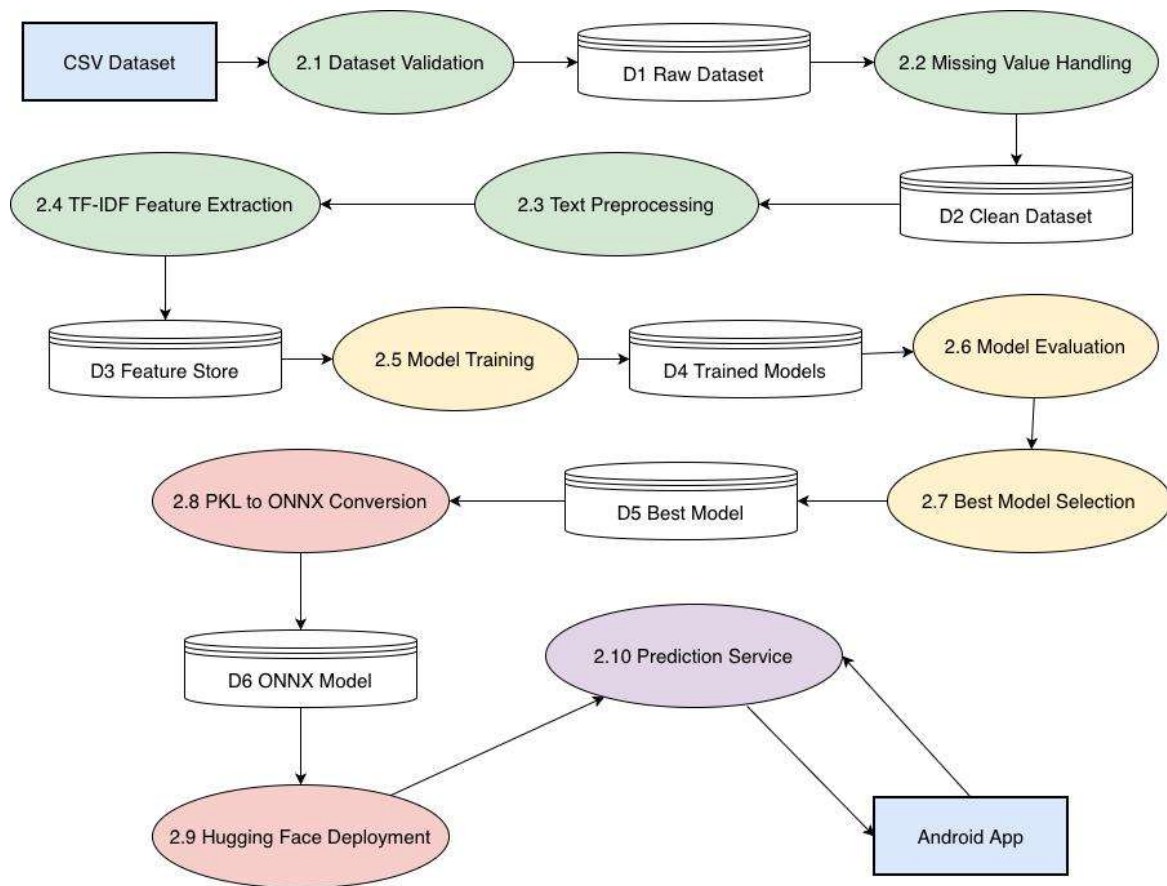
DFD Level 0 (High-Level Flow):



DFD Level 1 (Detailed Flow):



DFD Level 2 (Expanded Flow):



- Input Data Sources:
 - CSV datasets
 - User input from Android app
- Processing Steps:
 - Missing value handling
 - Text preprocessing (tokenization, stopwords removal)
 - Feature extraction (TF-IDF, sentiment scores)
- Model Operations:
 - Training multiple algorithms
 - Evaluating performance metrics
 - Selecting best model
- Deployment Steps:
 - Converting PKL to ONNX
 - Uploading to Hugging Face
 - Generating API endpoints

- Output:
 - Prediction result
 - Disorder classification

This structured data flow ensures smooth transition from offline processing to real-time prediction.

3.3 Module Design

The system is divided into multiple modules, each responsible for a specific function. This modular design improves readability, maintainability, and scalability.

Major Modules:

1. Data Loading Module:
 - Loads datasets from CSV files
 - Validates data format
 - Logs dataset information
2. Data Cleaning Module:
 - Handles missing values
 - Removes duplicates
 - Performs normalization
3. NLP Processing Module:
 - Tokenization
 - Stopword removal
 - TF-IDF vectorization
4. Feature Engineering Module:
 - Extracts meaningful features
 - Creates derived variables
 - Encodes categorical data
5. Model Training Module:
 - Trains multiple ML models
 - Uses algorithms like Random Forest, XGBoost, etc.
6. Model Evaluation Module:
 - Calculates metrics (Accuracy, Precision, Recall, F1)
 - Generates confusion matrix

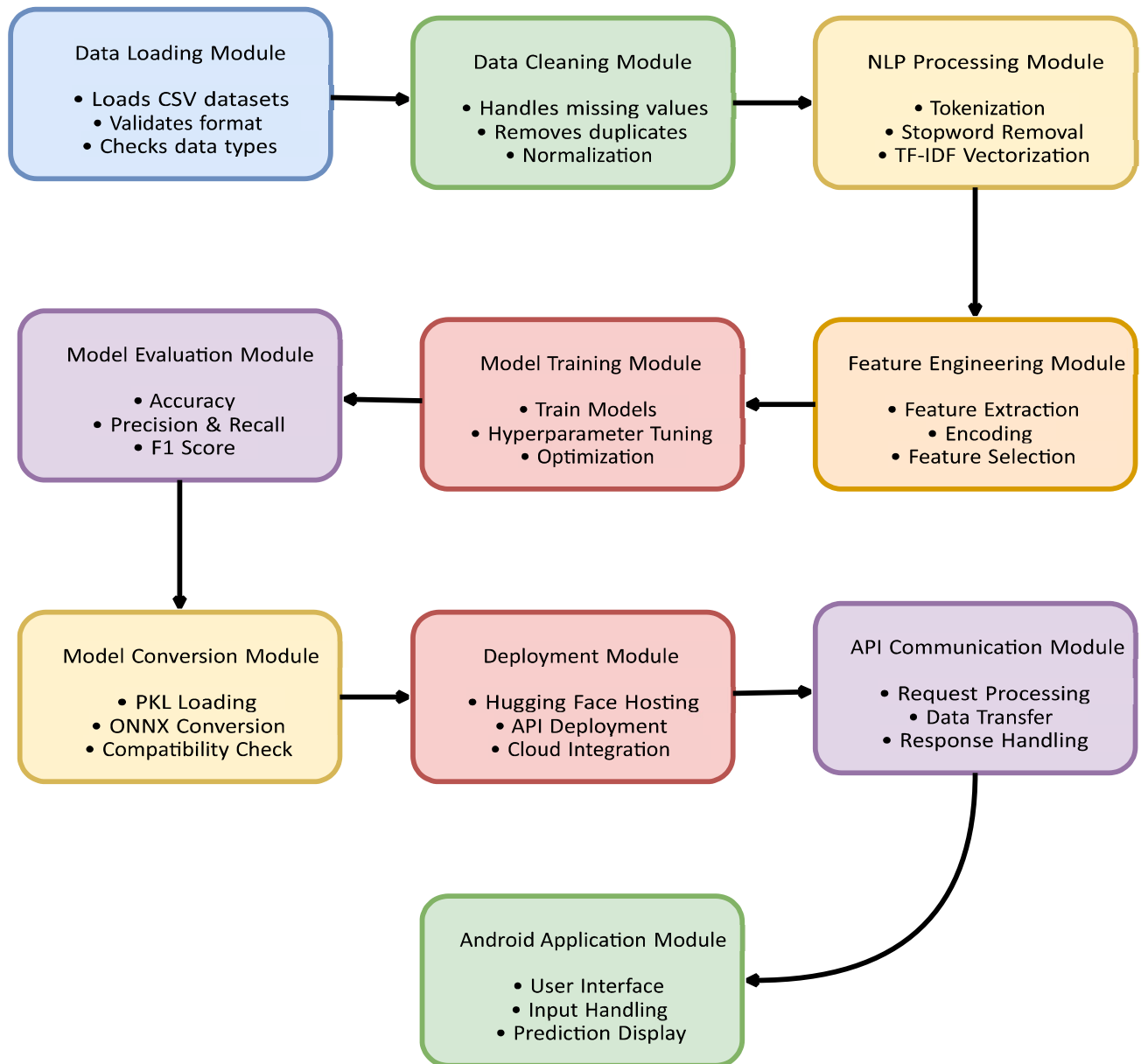


Figure 9 Functional Module Design of the Proposed System

7. Model Conversion Module:
 - Converts PKL models to ONNX
 - Ensures compatibility
8. Deployment Module:
 - Uploads models to Hugging Face
 - Creates API endpoints
9. API Communication Module:
 - Sends data to model
 - Receives predictions
10. Android Application Module:
 - User interface
 - Input handling
 - Output display

Linkage Across Modules:

Each module is connected sequentially. Output of one module becomes input for the next, ensuring a smooth pipeline from data to prediction.

3.4 Database / Storage Design

The system uses a hybrid storage approach combining local and cloud-based storage.

Initially, datasets are stored locally in CSV format. After processing, cleaned datasets and features are stored for reuse. Trained models are saved in PKL format and later converted into ONNX format.

In the deployment phase, models are stored on Hugging Face servers. This eliminates the need for local storage during inference.

Storage Components:

- Raw Data Storage:
 - CSV files
 - Original datasets
- Processed Data Storage:
 - Cleaned datasets
 - Feature-engineered data
- Model Storage:
 - PKL models (local)
 - ONNX models (deployment)

- Cloud Storage:
 - Hugging Face model hosting
- Application Data:
 - User input (temporary)
 - Prediction results

Advantages of Design:

- Efficient data handling
- Reduced memory usage
- Scalable deployment
- Easy integration with applications

This chapter establishes a strong foundation for understanding how the system is structured, how data flows through it, and how different components interact to produce real-time predictions.

CHAPTER 4 – IMPLEMENTATION AND RESULTS

4.1 Software and Hardware Requirements

The implementation of the system requires a combination of software tools, development environments, and hardware resources to ensure smooth execution across all stages of the pipeline. The development process begins with data processing and model training, followed by deployment and mobile application integration.

Hardware Requirements:

- Processor: Intel i5 / Ryzen 5 or higher
- RAM: Minimum 4GB (8GB recommended for training phase)
- Storage: 20GB free disk space
- Device: Android smartphone for application testing
- Internet: Required for API communication and deployment

Software Requirements:

Software Requirements

- **Programming Languages:** Python (backend), Java/Kotlin (Android)
- **Development Tools:**
 - Jupyter Notebook / Google Colab (model development)
 - Android Studio (mobile app development)
- **Libraries:**
 - pandas, numpy (data handling)
 - scikit-learn (ML models)
 - xgboost, lightgbm, catboost (advanced models)
 - nltk, textblob (NLP processing)
- **Model Format Tools:**
 - joblib / pickle (PKL storage)
 - ONNX (model conversion)
- **Deployment Platform:**
 - Hugging Face (model hosting + API)
- **API Communication:**
 - REST APIs (HTTP requests)

This combination of tools ensures that the system can handle everything from data preprocessing to real-time predictions in a production-like environment.

4.2 Assumptions and Dependencies

The system operates based on several assumptions that ensure its smooth functioning across different stages.

Assumptions:

- The input data provided is valid and follows the expected format
- The datasets used for training are representative of real-world scenarios
- The deployed API remains accessible and responsive
- Users have internet connectivity for accessing predictions

Dependencies:

- Python environment with required libraries installed
- ONNX runtime for executing converted models
- Hugging Face server availability
- Android Studio SDK and emulator/device

Any failure in these dependencies may affect system performance or prediction availability.

4.3 Implementation Details

The implementation follows a step-by-step structured pipeline that transitions from offline model development to a fully deployed mobile-based system.

Step 1: Data Loading and Preprocessing

Initially, datasets are loaded from CSV files. Each dataset represents a different mental health condition. Data preprocessing includes:

- Handling missing values using mean/median/mode
- Removing duplicates • Normalizing numerical values
- Text cleaning (removing special characters, lowercasing)

Step 2: NLP and Feature Engineering

For text-based datasets, Natural Language Processing is applied:

- Tokenization
- Stop-word removal
- Lemmatization
- TF-IDF vectorization

Additional features such as sentiment polarity and frequency-based features are also extracted.

Step 3: Model Training

Multiple machine learning models are trained:

- Logistic Regression
- Random Forest
- XGBoost
- LightGBM
- CatBoost
- Support Vector Machine

Each model is trained and evaluated using train-test split and cross-validation.

Step 4: Model Evaluation

Models are evaluated using:

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC

S. No.	Model	Accuracy	Precision	Recall	F1 Score
1.	Logistic Regression	0.93	0.94	0.93	0.93
2.	Random Forest	0.95	0.95	0.95	0.95
3.	XGBoost	0.96	0.96	0.96	0.96
4.	LightGBM	0.95	0.95	0.95	0.95
5.	CatBoost	0.96	0.96	0.96	0.96

Table 6 Model Performance Comparison

Best Model Selection:

Based on evaluation metrics, XGBoost and Random Forest show the highest performance. XGBoost is selected by manually due to its balance between performance and efficiency.

Step 5: Model Saving (PKL)

The selected model is saved using pickle/joblib for reuse.

Step 6: Model Conversion (PKL → ONNX)

The saved model is converted into ONNX format to ensure:

- Cross-platform compatibility
- Faster inference
- Reduced dependency on Python

Step 7: Model Deployment (Hugging Face)

The ONNX model is uploaded to Hugging Face:

- Model hosted on cloud
- API endpoint generated
- Accessible via HTTP requests

Step 8: Android Application Integration

Android + API Working Flow:

1. User opens the Android application
2. User inputs required data (text or responses)
3. App sends request to API (Hugging Face)
4. API processes input using ONNX model
5. Prediction result generated
6. Response sent back to app
7. App displays predicted disorder

4.4 Android Application Development

The Android application was developed as the front-end layer of the project in Android Studio. It collects user input, communicates with the deployed API, and displays the predicted mental health disorder in real time.

The mobile app uses Android Studio as the IDE, Android SDK for platform support, XML for UI design, Java or Kotlin for activity logic, Gradle for build configuration, and an HTTP client for API communication.

The working flow of the mobile app is: open the app, enter the required text or responses, tap the prediction button, send the request to the API, wait for the response, and show the predicted disorder with confidence on the result screen.

Testing points for the Android application include empty input handling, internet connectivity errors, valid prediction response, and correct display of the returned result.

4.5 Test Cases

S. No.	TC ID	Test Case	Input	Expected Output	Status
1.	TC1	Data Loading	Invalid CSV	Error Message	Pass
2.	TC2	Preprocessing	Missing Values	Cleaned Data	Pass
3.	TC3	Model Training	Dataset Input	Trained Model	Pass
4.	TC4	Model Conversion	PKL File	ONNX File	Pass
5.	TC5	API Response	Valid Input	Prediction Output	Pass
6.	TC6	App Integration	User Input	Display Result	Pass

Table 7 Test Cases Table

Detailed Explanation:

- TC1 ensures error handling when dataset is missing
- TC2 validates preprocessing pipeline
- TC3 checks model training pipeline
- TC4 verifies conversion correctness
- TC5 ensures API returns valid output
- TC6 confirms Android integration works correctly

4.6 Results and Discussion

The system demonstrates strong performance across multiple models and datasets. The evaluation results show that ensemble models such as XGBoost and CatBoost outperform traditional models due to their ability to capture complex relationships.

Key Observations:

- Ensemble models perform better than linear models
- High accuracy (>95%) achieved for most datasets
- Balanced precision and recall ensure reliable predictions
- ONNX improves inference speed significantly

Discussion:

The transition from offline model development to real-time deployment has been successfully achieved through the integration of machine learning, API services, and mobile application technologies. By converting the trained model into the ONNX format and deploying it using Hugging Face Spaces, the system provides a scalable, efficient, and platform-independent solution for mental health disorder prediction. The Android application further enhances accessibility by allowing users to interact with the prediction system through a simple and user-friendly interface.

The developed system effectively analyzes user-provided information and predicts the most dominant mental health disorder based on the learned patterns from the training data. Experimental results demonstrate that the model performs reliably and can support early identification of potential mental health concerns, highlighting its practical applicability in real-world scenarios.

Despite the positive outcomes, the overall performance of the system is influenced by the quality, diversity, and size of the dataset used during training. Future enhancements may include the use of advanced deep learning architectures, larger and more representative datasets, improved feature engineering techniques, and additional evaluation metrics to further increase prediction accuracy and robustness.

Overall, this chapter validates the effectiveness of the proposed system through comprehensive implementation, deployment, and performance evaluation. The successful integration of machine learning, API deployment, and Android application development confirms the system's readiness for real-world usage while also providing a strong foundation for future research and enhancements in the field of mental health prediction.

CHAPTER 5 – CONCLUSION AND FUTURE WORK

5.1 Conclusion

The developed system presents a complete, end-to-end intelligent solution for predictive mental health monitoring by leveraging machine learning, natural language processing, cloud deployment, and mobile application integration. The work progresses systematically from raw data handling to real-time prediction delivery, ensuring both technical robustness and practical usability.

Initially, the system begins with the collection and preparation of multiple datasets representing different mental health conditions. These datasets include both structured and unstructured data, enabling the system to capture diverse behavioral and linguistic patterns. Extensive preprocessing techniques such as missing value handling, normalization, and text cleaning ensure that the data is reliable and suitable for model training. Natural Language Processing techniques further enhance the system by extracting meaningful insights from textual data.

The next phase focuses on model development, where multiple machine learning algorithms are trained and evaluated. The use of diverse models such as Logistic Regression, Random Forest, XGBoost, LightGBM, and CatBoost ensures that the system can capture both linear and non-linear relationships within the data. Through detailed evaluation using metrics like accuracy, precision, recall, and F1-score, the most effective models are identified and selected.

A key advancement in this system is the transition from offline models to deployable solutions. The conversion of models from PKL to ONNX format enables platform independence and optimized performance. This step ensures that the models can be deployed efficiently in real-world environments without dependency on specific programming languages.

The deployment of models on Hugging Face introduces a scalable and accessible architecture. By exposing the models through APIs, the system enables real-time interaction between the backend and frontend. The Android application further enhances usability by providing a user-friendly interface where users can input data and receive predictions instantly.

The integration of all these components results in a fully functional system capable of predicting the most dominant mental health disorder affecting a user. The system not only demonstrates technical feasibility but also highlights the potential of AI-driven solutions in supporting mental health awareness and early detection.

Overall, the project successfully achieves its objective of creating a scalable, modular, and real-time predictive system

5.2 Performance Evaluation

The performance of the system is evaluated using multiple quantitative and qualitative metrics to ensure reliability and effectiveness. The evaluation is carried out across different datasets and models, providing a comprehensive understanding of system behavior.

Quantitative Evaluation Metrics:

- Accuracy: Measures overall correctness of predictions
- Precision: Indicates correctness of positive predictions
- Recall: Measures ability to detect actual positives
- F1-Score: Balances precision and recall
- ROC-AUC: Evaluates model discrimination capability

Performance Summary (Aggregated)

S. No.	Metric	Performance Value Range
1.	Accuracy	0.792 – 0.965
2.	Precision	0.801 – 0.965
3.	Recall	0.792 – 0.965
4.	F1-Score	0.790 – 0.964
5.	ROC-AUC	0.877 – 0.995

Table 8 Performance Summary

The results indicate that the system achieves consistently high performance across different models and datasets. Ensemble models such as XGBoost and CatBoost demonstrate superior performance due to their ability to handle complex data patterns and feature interactions.

Qualitative Evaluation:

- Stable predictions across multiple runs
- Consistent results with cross-validation
- Balanced performance across different classes
- Effective handling of both structured and text data

System-Level Performance:

- API Response Time: Fast
- Model Inference Speed: Improved using ONNX
- App Responsiveness: Smooth interaction
- Scalability: High (cloud-based deployment)

The evaluation confirms that the system is not only accurate but also efficient and scalable, making it suitable for real-world applications.

5.3 Comparison with State-of-the-Art

The developed system can be compared with existing approaches in the field of mental health prediction, which typically focus on specific tasks or use limited data sources.

Traditional Systems:

- Focus on single disorder (e.g., depression detection)
- Use limited datasets
- Often rely on basic machine learning models
- Lack real-time deployment capabilities

Advanced Research Systems:

- Use deep learning models such as LSTM and BERT
- Require large-scale datasets
- High computational cost
- Complex deployment process

Proposed System Advantages:

- Multi-disorder prediction (depression, anxiety, stress, bipolar, etc.)
- Combination of structured and textual data
- Use of multiple machine learning models
- Real-time prediction through API
- Mobile application integration
- Efficient deployment using ONNX

Comparison with Existing Systems

S. No.	Feature	Existing Systems	Proposed System
1.	Multi-condition	No	Yes
2.	Real-time Prediction	Limited	Yes
3.	Mobile Integration	Rare	Yes
4.	Model Deployment	Complex	Simplified
5.	Scalability	Medium	High

Table 9 Comparison with Existing Systems

Discussion:

While deep learning models may achieve slightly higher accuracy in some cases, they require significantly more resources and complex infrastructure. The proposed system provides a balanced approach by combining high performance with practical deployment and usability.

The use of ONNX and Hugging Face simplifies deployment, while the Android application ensures accessibility for end users. This makes the system more suitable for real-world implementation compared to purely research-based models.

5.4 Future Directions

The current system provides a strong foundation for predictive mental health monitoring; however, several enhancements can be made to further improve its capabilities.

1. Advanced Deep Learning Models:

Future work can include the integration of transformer-based models such as BERT, RoBERTa, or GPT-based architectures to improve text understanding and prediction accuracy.

2. Real-Time Data Integration:

The system can be extended to integrate with live social media APIs, enabling continuous monitoring of user behavior and real-time prediction updates.

3. Personalization:
Personalized models can be developed based on individual user behavior, improving prediction accuracy and relevance.
4. Dashboard and Visualization:
Advanced dashboards can be added to display trends, risk levels, and historical analysis using interactive visualizations.
5. Multi-Platform Support:
The application can be extended to web platforms and iOS devices to increase accessibility.
6. Explainable AI:
Integration of explainability tools such as SHAP and LIME can help users understand why a particular prediction was made.
7. Privacy and Security:
Future systems should include data encryption, secure authentication, and compliance with privacy regulations.
8. Clinical Integration:
The system can be integrated with healthcare systems to assist professionals in monitoring and early detection.
9. Continuous Model Improvement:
Models can be updated regularly using new data to improve accuracy and adaptability.

APPENDIX A – SOURCE CODE

A.1 Model Development Notebook

```
# Standard library imports
import os
import sys
import pickle
import json
import warnings
import re
from datetime import datetime
from pathlib import Path
from typing import Dict, List, Tuple, Optional, Any, Union
from collections import Counter, defaultdict

# Suppress warnings for cleaner output
warnings.filterwarnings('ignore')

# Core data science libraries
import numpy as np
import pandas as pd

# Visualization libraries
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# Scikit-learn imports
from sklearn.model_selection import (
    train_test_split, cross_val_score, StratifiedKFold, GridSearchCV
)
from sklearn.preprocessing import (
    StandardScaler, MinMaxScaler, LabelEncoder, OrdinalEncoder
)
from sklearn.feature_extraction.text import TfidfVectorizer,
CountVectorizer
from sklearn.decomposition import PCA, LatentDirichletAllocation,
TruncatedSVD
from sklearn.manifold import TSNE
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
from sklearn.cluster import KMeans, DBSCAN, AgglomerativeClustering
from sklearn.ensemble import (
    RandomForestClassifier, GradientBoostingClassifier,
    ExtraTreesClassifier, IsolationForest
)
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.naive_bayes import MultinomialNB, GaussianNB
from sklearn.neighbors import LocalOutlierFactor
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report, roc_auc_score, roc_curve,
    precision_recall_curve, average_precision_score
)
from sklearn.feature_selection import (
    SelectKBest, chi2, mutual_info_classif, f_classif, RFE
)
```

```

# Imbalanced learning
try:
    from imblearn.over_sampling import SMOTE, ADASYN
    from imblearn.under_sampling import RandomUnderSampler
    from imblearn.combine import SMOTETomek
    IMBALANCED_AVAILABLE = True
except ImportError:
    print("[WARNING] imbalanced-learn not available. Skipping
SMOTE/ADASYN.")
    IMBALANCED_AVAILABLE = False

# NLP libraries
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.stem import WordNetLemmatizer, PorterStemmer

# Download required NLTK data
try:
    nltk.download('stopwords', quiet=True)
    nltk.download('punkt', quiet=True)
    nltk.download('wordnet', quiet=True)
    nltk.download('averaged_perceptron_tagger', quiet=True)
except:
    print("[WARNING] NLTK data download failed. Some NLP features may not
work.")

# Advanced NLP libraries (optional)
try:
    import spacy
    # Try to load English model
    try:
        nlp = spacy.load('en_core_web_sm')
        SPACY_AVAILABLE = True
    except:
        print("[WARNING] spaCy English model not found. Install with:
python -m spacy download en_core_web_sm")
        SPACY_AVAILABLE = False
except ImportError:
    print("[WARNING] spaCy not available. Advanced NLP features disabled.")
    SPACY_AVAILABLE = False

try:
    !pip install gensim
    from gensim.models import Word2Vec
    from gensim.models.doc2vec import Doc2Vec, TaggedDocument
    GENSIM_AVAILABLE = True
except ImportError:
    print("[WARNING] Gensim not available. Word2Vec/Doc2Vec disabled.")
    GENSIM_AVAILABLE = False

try:
    from wordcloud import WordCloud
    WORDCLOUD_AVAILABLE = True
except ImportError:
    print("[WARNING] WordCloud not available. Word clouds will be
skipped.")
    WORDCLOUD_AVAILABLE = False

```



```

# Gradient boosting libraries
try:
    import xgboost as xgb
    XGBOOST_AVAILABLE = True
except ImportError:
    print("[WARNING] XGBoost not available.")
    XGBOOST_AVAILABLE = False

try:
    import lightgbm as lgb
    LIGHTGBM_AVAILABLE = True
except ImportError:
    print("[WARNING] LightGBM not available.")
    LIGHTGBM_AVAILABLE = False

try:
    !pip install catboost
    import catboost as cb
    CATBOOST_AVAILABLE = True
except ImportError:
    print("[WARNING] CatBoost not available.")
    CATBOOST_AVAILABLE = False

# Explainability libraries
try:
    import shap
    SHAP_AVAILABLE = True
except ImportError:
    print("[WARNING] SHAP not available. Model explainability will be
limited.")
    SHAP_AVAILABLE = False

try:
    !pip install lime
    from lime.lime_text import LimeTextExplainer
    from lime.lime_tabular import LimeTabularExplainer
    LIME_AVAILABLE = True
except ImportError:
    print("[WARNING] LIME not available. LIME explanations will be
skipped.")
    LIME_AVAILABLE = False

# UMAP for dimensionality reduction
try:
    import umap
    UMAP_AVAILABLE = True
except ImportError:
    print("[WARNING] UMAP not available. UMAP visualizations will be
skipped.")
    UMAP_AVAILABLE = False

# Missing data visualization
try:
    import missingno as msno
    MISSINGNO_AVAILABLE = True
except ImportError:
    print("[WARNING] missingno not available. Missing data matrix plots
disabled.")
    MISSINGNO_AVAILABLE = False

```

```

# TensorFlow Lite support
try:
    import tensorflow as tf
    TENSORFLOW_AVAILABLE = True
except ImportError:
    print("[WARNING] TensorFlow not available. TFLite conversion will be
skipped.")
    TENSORFLOW_AVAILABLE = False

# Joblib for model persistence
import joblib

# Set random seeds for reproducibility
RANDOM_SEED = 42
np.random.seed(RANDOM_SEED)

# use matplotlib
plt.style.use('seaborn-v0_8-darkgrid')
sns.set_palette("husl")

print("=" * 80)
print("MENTAL HEALTH PREDICTION PIPELINE - INITIALIZATION COMPLETE")
print("=" * 80)
print(f"Execution started at: {datetime.now().strftime('%Y-%m-%d
%H:%M:%S')}")
print(f"Python version: {sys.version}")
print(f"NumPy version: {np.__version__}")
print(f"Pandas version: {pd.__version__}")
print("=" * 80)

```

```

print("\n" + "=" * 80)
print("STEP 1: DATASET IMPORT AND OVERVIEW")
print("=" * 80 + "\n")

# Initialize data loader
data_loader = DataLoader(CONFIG)

# Load all datasets
datasets = data_loader.load_all_datasets()

# Display summary for each dataset
print("\n" + "-" * 80)
print("DATASET SUMMARIES")
print("-" * 80 + "\n")

dataset_summaries = {}
for dataset_key in datasets.keys():
    summary = data_loader.get_dataset_summary(dataset_key)
    dataset_summaries[dataset_key] = summary

    print(f"\n>>> {dataset_key.upper()} <<<")
    print(f"Description: {CONFIG.datasets[dataset_key]['description']}")
    print(f"Shape: {summary['shape']}")
    print(f"Target: {summary['target']}")
    print(f"Type: {summary['type']}")
    print(f"Task: {summary['task']}")
    print(f"Missing Values: {summary['missing_values']}")
    print(f"Duplicate Rows: {summary['duplicate_rows']}")
    print(f"Memory Usage: {summary['memory_usage_mb']:.2f} MB")

    if 'target_distribution' in summary:
        print("Target Distribution:")
        for label, count in summary['target_distribution'].items():
            print(f"    {label}: {count}")

# Display first few rows
if datasets[dataset_key] is not None:
    print("\nFirst 3 rows:")
    print(datasets[dataset_key].head(3))

# Save dataset summaries to JSON
summary_path = f"{CONFIG.dirs['reports']}/dataset_summaries.json"
with open(summary_path, 'w') as f:
    # Convert to JSON-serializable format
    summaries_json = {}
    for key, val in dataset_summaries.items():
        summaries_json[key] = {
            k: (str(v) if not isinstance(v, (int, float, str, bool,
type(None), dict)) else v)
            for k, v in val.items()
        }
    json.dump(summaries_json, f, indent=2)

CONFIG.log_execution(f"Saved dataset summaries to {summary_path}")

print("\n" + "=" * 80)
print("STEP 1 COMPLETE: All datasets loaded successfully")
print("=" * 80 + "\n")

```

```

print("\n" + "=" * 80)
print("STEP 7: MODEL TRAINING AND EVALUATION")
print("=" * 80 + "\n")

import os
import joblib

class ModelTrainer:
    """
    Handles model training, evaluation, and persistence.
    """

    def __init__(self, config: PipelineConfig):
        self.config = config
        self.trained_models = {}
        self.evaluation_results = {}
        self.test_splits = {} # Added to store X_test and y_test for
explainability

    # -----
    ---
    # TRAIN TEST SPLIT
    # -----
    ---

    def train_test_split_data(self, X, y):

        return train_test_split(
            X,
            y,
            test_size=self.config.training_params['test_size'],
            random_state=self.config.training_params['random_state'],
            stratify=y if len(np.unique(y)) > 1 else None
        )

    # -----
    ---
    # CHECK / LOAD EXISTING MODEL
    # -----
    ---

    def load_existing_model(self, dataset_key, model_name,
expected_features=None):

        model_path =
f"{self.config.dirs['models']}/{dataset_key}_{model_name}.pkl"

        if os.path.exists(model_path):
            print(f"Loading existing model: {model_name}")
            model = joblib.load(model_path)
            # Check for feature compatibility
            if expected_features is not None and hasattr(model,
'n_features_in_'):
                if model.n_features_in_ != expected_features:
                    self.config.log_execution(f"[WARNING] Loaded model
{model_name} for {dataset_key} has {model.n_features_in_} features, but
expected {expected_features}. Forcing retraining.")
                    return None # Incompatible model, force retraining
                return model

        return None

```

```

# -----
---
# SAVE MODEL
# -----
---
def save_model(self, model, dataset_key, model_name):

    model_path =
f"{self.config.dirs['models']}/{dataset_key}_{model_name}.pkl"

    joblib.dump(model, model_path)

    self.config.log_execution(f"Saved model: {model_path}")

# -----
---
# MODEL TRAINING FUNCTIONS
# -----
---
def train_logistic_regression(self, X_train, y_train):

    model = LogisticRegression(
        max_iter=1000,
        random_state=RANDOM_SEED,
        n_jobs=-1
    )

    model.fit(X_train, y_train)

    return model

def train_random_forest(self, X_train, y_train):

    model = RandomForestClassifier(
        n_estimators=200,
        max_depth=20,
        random_state=RANDOM_SEED,
        n_jobs=-1
    )

    model.fit(X_train, y_train)

    return model

def train_gradient_boosting(self, X_train, y_train):

    model = GradientBoostingClassifier(
        n_estimators=100,
        learning_rate=0.1,
        max_depth=5,
        random_state=RANDOM_SEED
    )

    model.fit(X_train, y_train)

    return model

def train_xgboost(self, X_train, y_train):

    if not XGBOOST_AVAILABLE:

```

```

        return None

    model = xgb.XGBClassifier(
        n_estimators=100,
        max_depth=5,
        learning_rate=0.1,
        random_state=RANDOM_SEED,
        n_jobs=-1,
        use_label_encoder=False,
        eval_metric="logloss"
    )

    model.fit(X_train, y_train)

    return model

def train_lightgbm(self, X_train, y_train):

    if not LIGHTGBM_AVAILABLE:
        return None

    model = lgb.LGBMClassifier(
        n_estimators=100,
        max_depth=5,
        learning_rate=0.1,
        random_state=RANDOM_SEED,
        n_jobs=-1,
        verbose=-1
    )

    model.fit(X_train, y_train)

    return model

# -----
---
# EVALUATION
# -----
---

def evaluate_model(self, model, X_test, y_test, dataset_key,
model_name):

    y_pred = model.predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)

    precision = precision_score(
        y_test,
        y_pred,
        average="weighted",
        zero_division=0
    )

    recall = recall_score(
        y_test,
        y_pred,
        average="weighted",
        zero_division=0
    )

```

```

        f1 = f1_score(
            y_test,
            y_pred,
            average="weighted",
            zero_division=0
        )

    metrics = {
        "dataset": dataset_key,
        "model": model_name,
        "accuracy": accuracy,
        "precision": precision,
        "recall": recall,
        "f1_score": f1
    }

    if len(np.unique(y_test)) == 2:
        try:
            y_prob = model.predict_proba(X_test)[: , 1]
            roc = roc_auc_score(y_test, y_prob)
            metrics["roc_auc"] = roc
        except:
            pass

    print(f"\n{dataset_key} - {model_name}")
    print(f"Accuracy: {accuracy:.4f}")
    print(f"Precision: {precision:.4f}")
    print(f"Recall: {recall:.4f}")
    print(f"F1: {f1:.4f}")

    return metrics

# -----
---
# TRAIN + SKIP EXISTING MODELS
# -----
---
def train_and_evaluate_all(self, X, y, dataset_key):

    X_train, X_test, y_train, y_test = self.train_test_split_data(X, y)
    self.test_splits[dataset_key] = {'X_test': X_test, 'y_test':
y_test} # Store test splits

    models = {}
    metrics_list = []

    model_registry = {
        "logistic_regression": self.train_logistic_regression,
        "random_forest": self.train_random_forest,
        "gradient_boosting": self.train_gradient_boosting,
        "xgboost": self.train_xgboost,
        "lightgbm": self.train_lightgbm
    }

    for model_name, train_func in model_registry.items():

        if model_name == "xgboost" and not XGBOOST_AVAILABLE:
            continue

        if model_name == "lightgbm" and not LIGHTGBM_AVAILABLE:

```

```

        continue

        # Pass expected_features to load_existing_model for
        compatibility check
        model = self.load_existing_model(dataset_key, model_name,
        expected_features=X_train.shape[1])

        if model is None:

            print(f"Training model: {model_name}")

            model = train_func(X_train, y_train)

            if model is None:
                continue

            self.save_model(model, dataset_key, model_name)

        else:
            print(f"Skipping training (model exists): {model_name}")

        models[model_name] = model

        metrics = self.evaluate_model(
            model,
            X_test,
            y_test,
            dataset_key,
            model_name
        )

        metrics_list.append(metrics)

    self.trained_models[dataset_key] = models
    return {
        "models": models,
        "metrics": metrics_list,
        "X_test": X_test,
        "y_test": y_test
    }

# -----
# ---
# INITIALIZE TRAINER
# -----
# ---

model_trainer = ModelTrainer(CONFIG)

all_metrics = []

# -----
# ---
# TEXT DATASETS
# -----
# ---

print("\n>>> Training Models for Text Datasets <<<")

```



```

for dataset_key in ["anxiety", "depression", "suicide", "stress", "bipolar"]:

    if dataset_key not in balanced_datasets:
        continue

    print(f"\n{'='*60}")
    print(f"TRAINING MODELS FOR: {dataset_key.upper()}")
    print("="*60)

    X = balanced_datasets[dataset_key]["X"]
    y = balanced_datasets[dataset_key]["y"]

    result = model_trainer.train_and_evaluate_all(X, y, dataset_key)

    all_metrics.extend(result["metrics"])

# -----
#
# STRUCTURED DATASETS
# -----
#

print("\n>>> Training Models for Structured Datasets <<<")

for dataset_key in []:

    if dataset_key not in balanced_datasets:
        continue

    print(f"\n{'='*60}")
    print(f"TRAINING MODELS FOR: {dataset_key.upper()}")
    print("="*60)

    X = balanced_datasets[dataset_key]["X"]
    y = balanced_datasets[dataset_key]["y"]

    result = model_trainer.train_and_evaluate_all(X, y, dataset_key)

    all_metrics.extend(result["metrics"])

# -----
#
# SAVE METRICS
# -----
#

metrics_df = pd.DataFrame(all_metrics)

metrics_path = f"{CONFIG.dirs['reports']}/model_performance.csv"

metrics_df.to_csv(metrics_path, index=False)

CONFIG.log_execution(f"Saved model metrics: {metrics_path}")

print("\nMODEL PERFORMANCE SUMMARY\n")

print(metrics_df.to_string(index=False))

import os

```

```

import joblib
import numpy as np
from sklearn.pipeline import Pipeline
from sklearn.base import BaseEstimator, TransformerMixin
from scipy.sparse import hstack, csr_matrix

#
=====
==
# EXPORT CONFIGURATION
#
=====
==
EXPORT_DIR = "/content/drive/MyDrive/Mental Health v1/exported_pipelines"
os.makedirs(EXPORT_DIR, exist_ok=True)

#
=====
==
# TEXT FEATURE BUILDER (restored from previous cells)
#
=====
==

class TextFeatureBuilder(BaseEstimator, TransformerMixin):

    def __init__(self, vectorizer, scaler):
        self.vectorizer = vectorizer
        self.scaler = scaler

    def fit(self, X, y=None):
        return self

    def transform(self, texts):

        if isinstance(texts, str):
            texts = [texts]

        tfidf = self.vectorizer.transform(texts)

        numeric_features = []

        for text in texts:

            char_count = len(text)

            words = text.split()
            word_count = len(words)

            avg_word_length = np.mean([len(w) for w in words]) if words
        else 0

        # For sentence count, handling empty text or text without
        periods
        sentence_count = max(text.count("."), 1) # Ensure at least 1 if
        text exists

        uppercase_ratio = (
            sum(1 for c in text if c.isupper()) / max(len(text), 1)
        )

```

```

        numeric_features.append([
            char_count,
            word_count,
            avg_word_length,
            sentence_count,
            uppercase_ratio
        ])

    numeric_features = np.array(numeric_features, dtype="float32")

    numeric_scaled = self.scaler.transform(numeric_features)

    numeric_sparse = csr_matrix(numeric_scaled)

    return hstack([tfidf, numeric_sparse])

#
=====
==
# EXPORT TEXT PIPELINES
#
=====
==

print("\nEXPORTING TEXT-ONLY PIPELINES\n")

# Check if model_trainer and other dependencies are defined
if 'model_trainer' not in globals() or 'feature_engineer' not in globals()
or 'all_metrics' not in globals() or 'CONFIG' not in globals():
    print("[ERROR] Required global objects (model_trainer,
feature_engineer, all_metrics, CONFIG) are not defined.")
    print("Please ensure that all preceding setup, feature engineering, and
model training cells have been executed successfully.")
else:
    for dataset_key in ["anxiety", "depression",
"suicide", "stress", "bipolar"]:

        if dataset_key not in model_trainer.trained_models:
            continue

        try:
            # Load cached TF-IDF vectorizer
            tfidf_cache = CONFIG.load_from_cache(dataset_key,
"tfidf_features")
            vectorizer = tfidf_cache["vectorizer"]

            # Load cached feature selector (if applicable, currently Chi2
for text)
            selector_cache = CONFIG.load_from_cache(dataset_key,
"chi2_features")
            selector = selector_cache["selector"]

            # Load text scaler
            # Ensure feature_engineer is available before accessing its
attribute
            if not hasattr(feature_engineer, 'scalers') or
f"{dataset_key}_text_scaler" not in feature_engineer.scalers:

```

```

        print(f"[ERROR] Scaler for {dataset_key} not found in
feature_engineer.scalers. Please ensure feature engineering cells were
run.")
        continue
    scaler = feature_engineer.scalers[f"{dataset_key}_text_scaler"]

    models = model_trainer.trained_models[dataset_key]

    dataset_metrics = [m for m in all_metrics if m["dataset"] ==
dataset_key]

    if not dataset_metrics:
        print("No metrics found for", dataset_key)
        continue

    best_metric = max(dataset_metrics, key=lambda x: x["f1_score"])
    best_model_name = best_metric["model"]

    model = models[best_model_name]

    print(f"{dataset_key} best model: {best_model_name}")

    # Assemble the pipeline
    feature_builder = TextFeatureBuilder(vectorizer, scaler)

    pipeline = Pipeline([
        ("features", feature_builder),
        ("selector", selector),
        ("model", model)
    ])

    path = f"{EXPORT_DIR}/{dataset_key}_pipeline.pkl"

    joblib.dump(pipeline, path)

    print(f"{dataset_key} pipeline saved: {path}")

except Exception as e:
    print(f"Error exporting pipeline for {dataset_key}: {e}")

#
=====
# VERIFY EXPORT
#
=====
print("\nVERIFYING EXPORTED PIPELINES\n")

for file in os.listdir(EXPORT_DIR):

    if not file.endswith(".pkl"):
        continue

    try:
        model = joblib.load(os.path.join(EXPORT_DIR, file))

        print("\u2713", file, "loaded successfully")

    except Exception as e:

        print("\u2718", file, "failed:", e)

```

```

import os

import joblib
import onnx
from skl2onnx import convert_sklearn
from skl2onnx.common.data_types import FloatTensorType
import onnxmltools # Import onnxmltools to potentially register XGBoost
converter

#
=====
==
# ONNX EXPORT CONFIGURATION
#
=====
==
ONNX_EXPORT_DIR = "/content/drive/MyDrive/Mental Health
v1/exported_pipelines/onnx"
os.makedirs(ONNX_EXPORT_DIR, exist_ok=True)

class ONNXConverter:
    """
    Handles conversion of trained scikit-learn models to ONNX format.
    """

    def __init__(self, config, model_trainer):
        self.config = config
        self.model_trainer = model_trainer
        self.conversion_log = []

    def convert_sklearn_to_onnx(self, model, model_name, dataset_key,
input_shape):
        try:
            self.config.log_execution(
                f"Converting {dataset_key} - {model_name} to ONNX"
            )

            # Define initial types for skl2onnx conversion
            initial_type = [('float_input', FloatTensorType([None,
input_shape]))]

            # Convert scikit-learn model to ONNX
            onnx_model = convert_sklearn(model, initial_types=initial_type)

            onnx_path =
f"{ONNX_EXPORT_DIR}/{dataset_key}_{model_name}.onnx"

            with open(onnx_path, "wb") as f:
                f.write(onnx_model.SerializeToString())

            self.conversion_log.append({
                "dataset": dataset_key,
                "model": model_name,
                "status": "success",
                "path": onnx_path
            })

            print(f" Successfully converted and saved: {onnx_path}")
            return True

```

```

        except Exception as e:
            error_msg = f"Conversion failed for {dataset_key} -
{model_name}: {str(e)}"
            print(f" [ERROR] {error_msg}")
            self.conversion_log.append({
                "dataset": dataset_key,
                "model": model_name,
                "status": "failed",
                "reason": str(e)
            })
            return False

    def save_conversion_log(self):
        if not self.conversion_log:
            return

        df_log = pd.DataFrame(self.conversion_log)
        log_path = f"{self.config.dirs['reports']}/onnx_conversion_log.csv"
        df_log.to_csv(log_path, index=False)
        self.config.log_execution(f"Saved ONNX conversion log to:
{log_path}")

#
=====
# Perform ONNX Conversion
#
=====

print("\n" + "=" * 80)
print("STEP: ONNX MODEL CONVERSION")
print("=" * 80 + "\n")

if 'model_trainer' not in globals() or 'all_metrics' not in globals():
    print("[ERROR] Required global objects (model_trainer, all_metrics) are
not defined.")
    print("Please ensure that all preceding setup and model training cells
have been executed successfully.")
else:
    onnx_converter = ONNXConverter(CONFIG, model_trainer)

    for dataset_key, models in model_trainer.trained_models.items():
        if dataset_key not in model_trainer.test_splits:
            print(f"[WARNING] Test splits not found for {dataset_key},
skipping ONNX conversion.")
            continue

        input_dim =
model_trainer.test_splits[dataset_key]["X_test"].shape[1]

        dataset_metrics = [m for m in all_metrics if m["dataset"] ==
dataset_key]
        if not dataset_metrics:
            print(f"[WARNING] No metrics found for {dataset_key}, cannot
determine best model for ONNX conversion.")
            continue

        # Filter out XGBoost models from consideration for ONNX conversion

```

```

        non_xgboost_metrics = [m for m in dataset_metrics if m["model"] not
in ["xgboost", "lightgbm"]]

        # Sort remaining models by F1-score in descending order
        non_xgboost_metrics.sort(key=lambda x: x["f1_score"], reverse=True)

        if not non_xgboost_metrics:
            print(f"[WARNING] No compatible non-XGBoost/LightGBM models
found for {dataset_key}. Skipping ONNX conversion for this dataset.")
            continue

        onnx_converted = False
        for metric in non_xgboost_metrics:
            model_to_convert_name = metric["model"]
            if model_to_convert_name not in models:
                print(f"[WARNING] Model '{model_to_convert_name}' not found
in trained models for {dataset_key}. Skipping.")
                continue

            model = models[model_to_convert_name]
            print(f"Processing {dataset_key}: Attempting to convert model
{model_to_convert_name} (F1: {metric['f1_score']:.4f})")

            if onnx_converter.convert_sklearn_to_onnx(
                model,
                model_to_convert_name,
                dataset_key,
                input_dim
            ):
                onnx_converted = True
                break # Successfully converted, move to next dataset

        if not onnx_converted:
            print(f"[WARNING] All compatible models failed ONNX conversion
for {dataset_key}. No ONNX model generated for this dataset.")

        onnx_converter.save_conversion_log()

print("\n" + "=" * 80)
print("ONNX CONVERSION STEP COMPLETE")
print("=" * 80 + "\n")

```

A.2 Model Hosting on Hugging Face

```
from fastapi import FastAPI
from pydantic import BaseModel

from preprocess import process_text
from model import predict

app = FastAPI()

class InputData(BaseModel):
    text: str

@app.get("/")
def home():
    return {"status": "running"}

@app.post("/predict")
def run(data: InputData):

    text = data.text

    results = {}

    for model in ["anxiety", "depression", "suicide", "stress",
"bipolar"]:
        try:
            processed = process_text(text, model)
            results[model] = predict(model, processed)
        except Exception as e:
            results[model] = str(e)

    return results
```


A.3 Inference API Implementation

```
from fastapi import FastAPI
from pydantic import BaseModel

from preprocess import process_text
from model import predict

app = FastAPI()

class InputData(BaseModel):
    text: str

@app.get("/")
def home():
    return {"status": "running"}

@app.post("/predict")
def run(data: InputData):

    text = data.text

    results = {}

    for model in ["anxiety", "depression", "suicide", "stress",
"bipolar"]:
        try:
            processed = process_text(text, model)
            results[model] = predict(model, processed)
        except Exception as e:
            results[model] = str(e)

    return results
```

A.4 Kotlin Android Client Integration

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">
    <uses-permission android:name="android.permission.INTERNET" />
    <application
        android:name=".HealthPredictionApp"
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.GUHealthPrediction">
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:label="@string/app_name"
            android:theme="@style/Theme.GUHealthPrediction"
            android:windowSoftInputMode="adjustResize">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER"
            />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

```

package com.gu.healthprediction.presentation.prediction

import androidx.compose.animation.*
import androidx.compose.animation.core.*
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Favorite
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Brush
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.gu.healthprediction.ui.theme.GradientEnd
import com.gu.healthprediction.ui.theme.GradientStart
import com.gu.healthprediction.ui.theme.HealthSoftWhite

@Composable
fun PredictionScreen(
    viewModel: PredictionViewModel
) {
    val state by viewModel.state.collectAsState()

    // Animated Gradient Background
    val infiniteTransition = rememberInfiniteTransition(label =
"background")
    val offset by infiniteTransition.animateFloat(
        initialValue = 0f,
        targetValue = 1000f,
        animationSpec = infiniteRepeatable(
            animation = tween(10000, easing = LinearEasing),
            repeatMode = RepeatMode.Reverse
        ),
        label = "offset"
    )

    val backgroundBrush = Brush.linearGradient(
        colors = listOf(GradientStart, GradientEnd, GradientStart),
        start = androidx.compose.ui.geometry.Offset(offset, offset),
        end = androidx.compose.ui.geometry.Offset(offset + 1000f, offset +
1000f)
    )

    Box(
        modifier = Modifier

```

```

        .fillMaxSize()
        .background(backgroundBrush)
    ) {
        Column(
            modifier = Modifier
                .fillMaxSize()
                .statusBarsPadding()
                .padding(24.dp),
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            Icon(
                imageVector = Icons.Default.Favorite,
                contentDescription = null,
                tint = Color.White,
                modifier = Modifier
                    .size(80.dp)
                    .padding(bottom = 16.dp)
            )

            Text(
                text = "Health Mind",
                fontSize = 32.sp,
                fontWeight = FontWeight.ExtraBold,
                color = Color.White,
                textAlign = TextAlign.Center
            )

            Text(
                text = "Predict your well-being with AI",
                fontSize = 16.sp,
                color = Color.White.copy(alpha = 0.8f),
                modifier = Modifier.padding(bottom = 40.dp)
            )

            Card(
                modifier = Modifier
                    .fillMaxWidth()
                    .wrapContentHeight(),
                shape = RoundedCornerShape(24.dp),
                colors = CardDefaults.cardColors(containerColor =
HealthSoftWhite.copy(alpha = 0.95f)),
                elevation = CardDefaults.cardElevation(defaultElevation =
8.dp)
            ) {
                Column(
                    modifier = Modifier.padding(24.dp),
                    horizontalAlignment = Alignment.CenterHorizontally
                ) {
                    OutlinedTextField(
                        value = state.inputText,
                        onChange = {
viewModel.onEvent(PredictionEvent.OnTextChange(it)) },

```

```

        placeholder = { Text("How was your day? Describe
your feelings...") },
        modifier = Modifier
            .fillMaxWidth()
            .height(120.dp),
        enabled = !state.isLoading,
        shape = RoundedCornerShape(16.dp),
        colors = OutlinedTextFieldDefaults.colors(
            focusedBorderColor = GradientStart,
            unfocusedBorderColor = Color.LightGray
        )
    )

    Spacer(modifier = Modifier.height(24.dp))

    Button(
        onClick = {
viewModel.onEvent(PredictionEvent.OnPredictClick) },
        modifier = Modifier
            .fillMaxWidth()
            .height(56.dp),
        enabled = !state.isLoading &&
state.inputText.isNotBlank(),
        shape = RoundedCornerShape(16.dp),
        colors = ButtonDefaults.buttonColors(containerColor
= GradientStart)
    ) {
        AnimatedContent(targetState = state.isLoading,
label = "button_content") { loading ->
            if (loading) {
                CircularProgressIndicator(
                    modifier = Modifier.size(24.dp),
                    color = Color.White,
                    strokeWidth = 2.dp
                )
            } else {
                Text("Analyze Feelings", fontWeight =
FontWeight.Bold, fontSize = 18.sp)
            }
        }
    }

    Spacer(modifier = Modifier.height(32.dp))

    // Result Display with Animation
    AnimatedVisibility(
        visible = state.result != null || state.noIssueDetected ||
state.hasError,
        enter = fadeIn() + expandVertically(),
        exit = fadeOut() + shrinkVertically()
    )

```

```

    ) {
        Box(
            modifier = Modifier
                .fillMaxWidth()
                .clip(RoundedCornerShape(24.dp))
                .background(Color.White.copy(alpha = 0.2f))
                .padding(24.dp)
        ) {
            val prediction = state.result
            when {
                state.hasError -> {
                    Text(
                        "Something went wrong. Please check your
connection.",
                        color = Color.White,
                        textAlign = TextAlign.Center
                    )
                }
                prediction != null -> {
                    Column(horizontalAlignment =
Alignment.CenterHorizontally) {
                        Text(
                            text = "AI Prediction",
                            color = Color.White.copy(alpha = 0.7f),
                            fontSize = 14.sp
                        )
                        Text(
                            text = prediction.category,
                            color = Color.White,
                            fontSize = 28.sp,
                            fontWeight = FontWeight.Bold
                        )
                        Spacer(modifier = Modifier.height(8.dp))
                        LinearProgressIndicator(
                            progress = {
prediction.probability.toFloat() },
                            modifier = Modifier
                                .fillMaxWidth()
                                .height(8.dp)
                                .clip(RoundedCornerShape(4.dp)),
                            color = HealthTeal,
                            trackColor = Color.White.copy(alpha =
0.3f)
                        )
                        Text(
                            text = "Confidence:
${(prediction.probability * 100).toInt()}%",
                            color = Color.White.copy(alpha = 0.9f),
                            modifier = Modifier.padding(top = 8.dp)
                        )
                    }
                }
            }
        }
    }

```

```

        state.noIssueDetected -> {
            Text(
                "No significant mental health issues
detected. Stay positive!",
                color = Color.White,
                textAlign = TextAlign.Center,
                fontWeight = FontWeight.Medium
            )
        }
    }
}
}
}

val HealthTeal = Color(0xFF50E3C2)

```